Table of Contents

1.INTRODUCTION TO TESTING....1
1.1. TESTING....1
1.2.IMPORTANCE OF SOFTWARE TESTING....2
1.3.VERIFICATION AND VALIDATION....2
1.4.TESTING LIFE CYCLE....3
1.5.UNDERSTANDING TESTING PROCESS....5
2.CATEGORIES OF TESTING....8
2.1.CATEGORIES OF TESTING....8
2.2.DYNAMIC TESTING TECHNIQUES....9
a)White Box Testing....9
b)Black Box Testing....9
2.3.BLACK BOX TESTING TECHNIQUES....10
a)Equivalence Partitioning....10
b) Boundary Value Analysis....10
2.4.WHITE BOX TESTING TECHNIQUES....11
a) Statement Coverage....11
b) Branch or Decision Coverage....11
c)Multiple Condition Coverage....11
d) Loop Coverage....11
e)Call Coverage....11
f)Path Coverage....11
2.5.LEVELS OF TESTING....11
2.6.UNIT TESTING....12
2.7.INTEGRATION TESTING....13
2.8.SYSTEM TESTING....13
2.9.ACCEPTANCE TESTING....14
2.10.NON FUNCTIONAL TESTING....14
a)Performance testing:....15
b)Volume Testing:....15
c)Load testing:....16
d)Stress Testing:....16
e)Disaster Testing....16
f)Recovery Testing:....17
g)Security Testing:....17
h)Reliability testing:....17
i)Installation Testing:....18
j)Usability Testing:....18
k)Accessibility Testing:....19
l)Regression testing:....19

1. INTRODUCTION TO TESTING

1.1. TESTING

Testing is the process of executing a program with the intent of finding errors

1.2. IMPORTANCE OF SOFTWARE TESTING

As we all are human beings and human beings commit errors in any process, some of the errors do not impact much on our day to day life and can be ignored, however some errors are so severe that they can break the whole system or software. In such kind of situations you need to take care that such errors are caught well in advance before deploying the system/software in production environment.

Lets take some real life examples and see how our errors can cause critical failures or disasters if not caught early.

Examples:
Lets take some ABC banks net banking website, if any net banking customer logs into the website and transfers some amount to other account. After transferring he got the confirmation that amount got transferred successfully and it got deducted from his account. After sometime when he confirmed with other person whom he transferred the amount he got to know that he has not received the amount, now you need to visit your bank to settle this dispute. Also you will get annoyed by the net banking experience of that bank.

This is one situation where testing your software is very important before deploying it. Testing cannot be ignored because it impacts all the end users of software.

If the website would have got tested thoroughly for all the possible user operations this problem would have been found well in advance and got fixed before deploying the website.

Lets take another example.
Lots of software are nowadays used in air traffic control system. Suppose we don't test air traffic control software and directly deploy them in production environment, there is very much risk involved in this, most probably this software will fail sometime because it has not been tested for all the scenarios in which it should work. If the air traffic software crashes it may lead to very severe disaster.

Now you might have got an idea why testing is such an important process in any kind of product development.

You can think of few more real life examples where testing should be done before deploying the product in production environment.

1.3. VERIFICATION AND VALIDATION

Verification

It is to establish the truth of correspondence between a software product and its specification.
ex: Are we building the product right?

Validation

It is to establish the fitness or worth of a software product for its operational mission
ex: Are we building the right product?

1.4. TESTING LIFE CYCLE

Software Testing Life Cycle process is an integral part of the Software Development Life Cycle. The overall aspect of STLC phase deals with testing and rectifying any error code generating within the program under various test conditions.

STLC is simply a testing phase in the SDLC development. Validation and Authentication is tried and tested in this phase. The only limitation of this cycle is that it is limited to respective individual phase and is carried out by a group of skilled testers and technology evangelistic

Testing Life Cycle consists of the following steps as stated below:

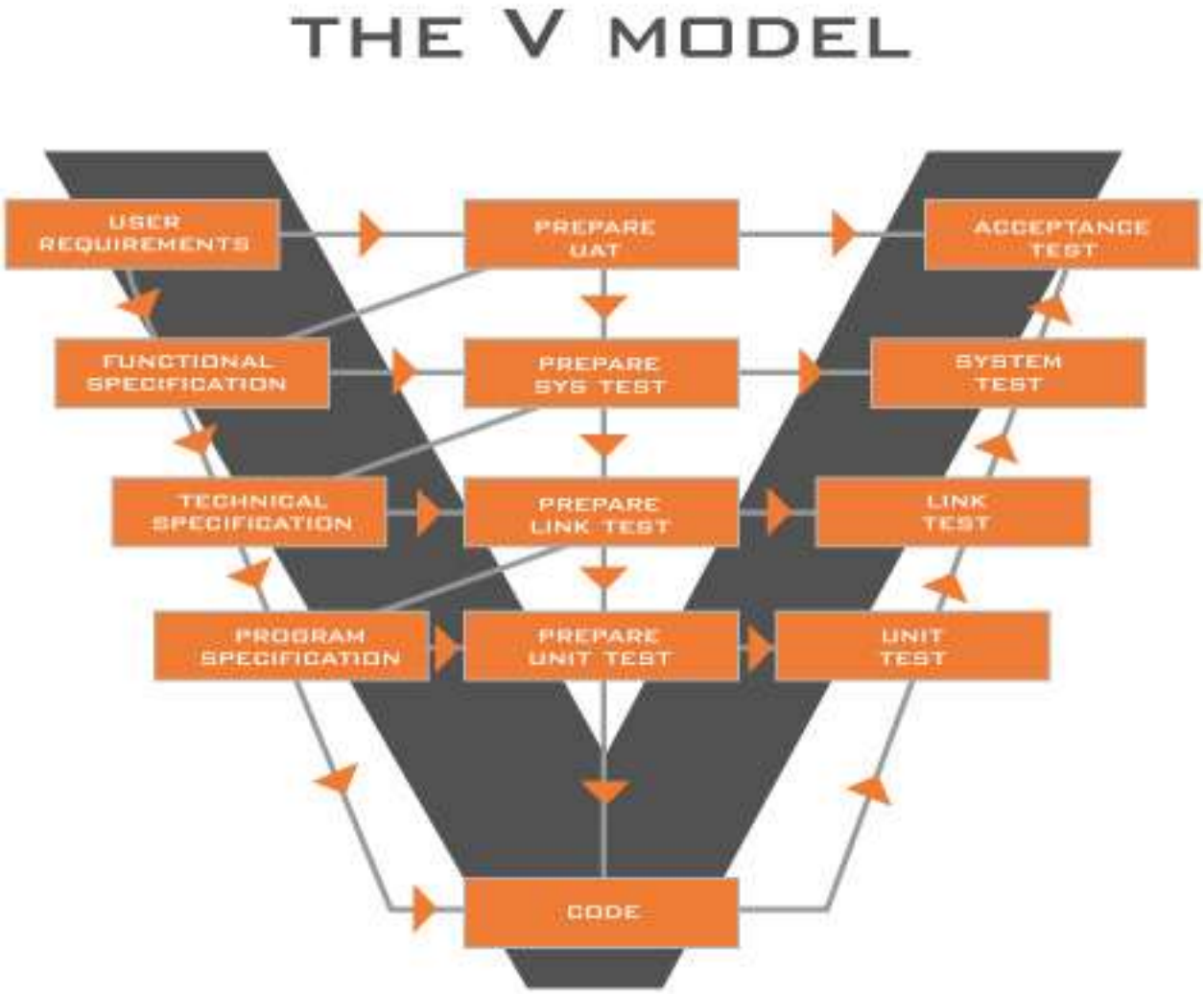
Requirements/Design Review: You review the software requirements/design (Well, if they exist.)
Test Planning: Once you have gathered a general idea of what needs to be tested, you 'plan' for the tests.
Test Designing: You design/detail your tests on the basis of detailed requirements/design of the software (sometimes, on the basis of your imagination).
Test Environment Setup: You setup the test environment (server/client/network, etc) with the goal of replicating the end-users' environment.
Test Execution: You execute your Test Cases/Scripts in the Test Environment to see whether they pass.
Test Reporting: You prepare various reports for various stakeholders.

1.5. *UNDERSTANDING TESTING PROCESS*

Test Process Management (TPM) is concerned with controlling all the testing activities and the automated support tools used in a project, within a dedicated management environment. TPM is based upon a professionally recognised industry standard - 'The V Model', which supports each stage of the system development life cycle. The V-Model demonstrates the complexity of relationships between each stage of the development life cycle and acknowledges that for every stage within the development there is an associated stage of testing.
The conventional V model was suited for Waterfall software development process. Iterative V Model is what we follow at HyTech Professionals. Iterative test process is best suitable for projects where development is done by one team and testing is done by an independent team.

Iterative test process consists of many iteration and every iteration consists of phases.
Each iteration has one release and all phases of V model are executed in one iteration. This leads to better quality release as changes done in one iteration are tested and complete system is tested again for

any break point that crept in because of implementation in current iteration. In every iteration, percentage activity of phase is decided based on specification change scope.

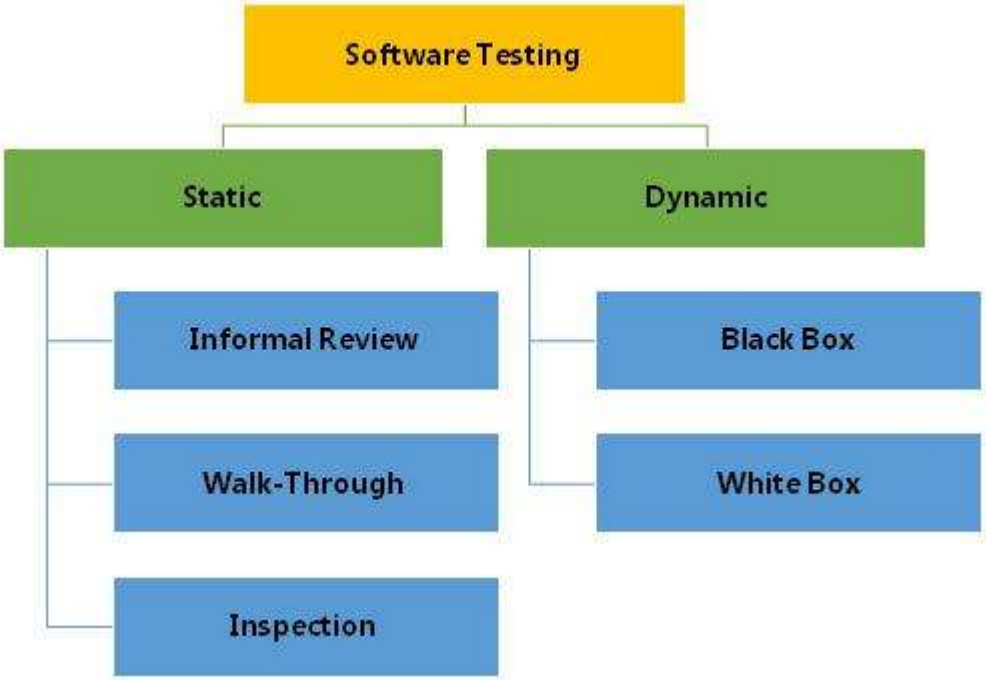


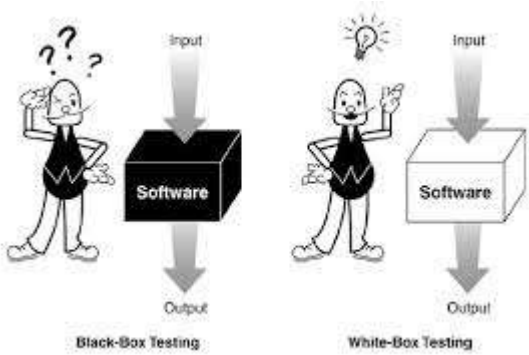
The objective is to ensure every element of the system is validated at the earliest possible stage, to the quality criteria set out by the business managers, providing a comprehensible and manageable audit trail of the systems actual capabilities.

The benefits of this level of control and management over quality assurance are compounded when changes are introduced. With a manual system, checking the impact of even minor changes is complex, time consuming and prone to human error. With T-Plan this is a simple 'What if?' function of reporting impacts of the proposed changes at both technical and business levels. This allows the risk of potential error/fault correction to be identified, together with potential for 'knock-on' or efficiency costs to the business.

2. CATEGORIES OF TESTING

2.1. CATEGORIES OF TESTING.





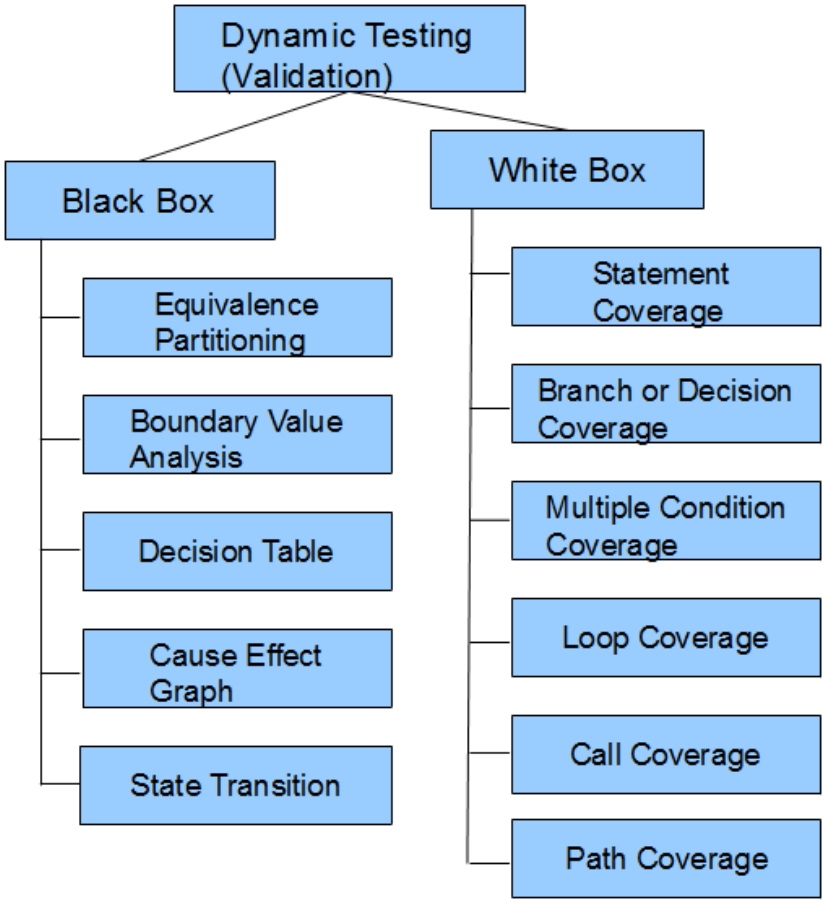
2.2. DYNAMIC TESTING TECHNIQUES.

White Box Testing

Tests at micro level of the programs that test each and every implemented functional task, ensuring that all code options are exercised. Requires knowledge of the internal code.

b) Black Box Testing

Testing that focuses solely on the outputs generated in response to selected inputs and execution conditions. Requirements are the only test basis and knowledge of the internal code is not required.



2.3. BLACK BOX TESTING TECHNIQUES.

a) Equivalence Partitioning

Partitioning the input domain of a program into a finite number of classes (sets), to identify a minimal set of well selected test cases to represent these classes.

There are two types of input equivalence classes, valid and invalid.

Equivalence class testing can significantly reduce the number of test cases that must be created

and executed

b) Boundary Value Analysis

A selection technique in which test data are chosen to lie along "boundaries“ of the input domain [or output range] classes, data structures, procedure parameters, etc.

Choices often include maximum, minimum, and trivial values

2.4. WHITE BOX TESTING TECHNIQUES.

a) Statement Coverage

Testing to satisfy the criterion that each statement in a program to be executed at least once during program testing. Coverage is 100 percentage when a set of test cases causes every program statement to be executed at least once.

The chief disadvantage of statement coverage is that it is insensitive to some control structures.

b) Branch or Decision Coverage

A test coverage criteria which requires that for each decision point or each possible branch be executed at least once

c) Multiple Condition Coverage

A test coverage criteria which requires enough test cases such that all possible combinations of condition outcomes in each decision, and all points of entry, are invoked at least once.

A large number of test cases may be required for full multiple condition coverage

d) Loop Coverage

A test coverage criteria which checks whether loop body executed zero times, exactly once or more than once

e) Call Coverage

A test coverage criteria which checks whether function called zero times, exactly once or more than once

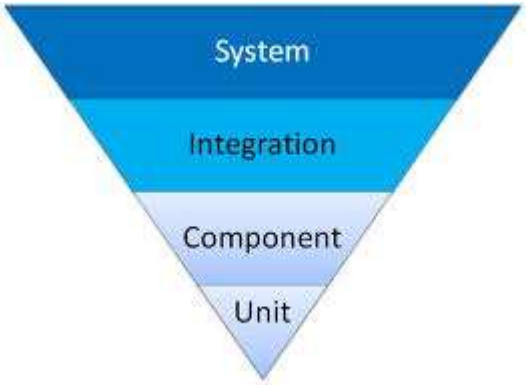
Since probability of failure is more in function calls, each function call is executed

f) Path Coverage

Testing to satisfy coverage criteria that each logical path through the program be tested. Often paths through the program are grouped into a finite set of classes. One path from each class is then tested

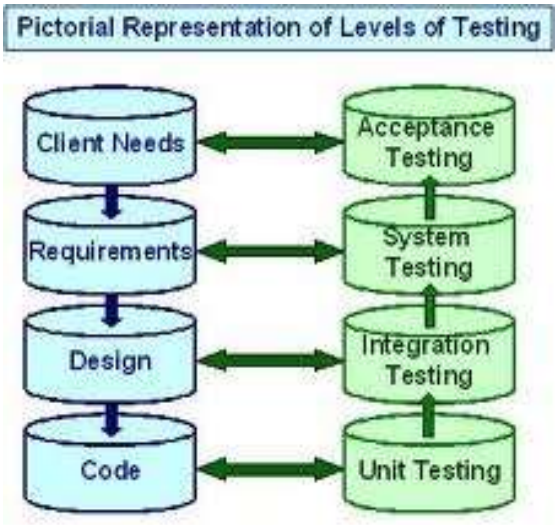
General coverage requires executing all paths, number of paths may be infinite if there are loops

2.5. *LEVELS OF TESTING*



There are broadly four levels of Testing done in any Testing Project

- ✓ Unit Testing
- ✓ Integration Testing
- ✓ System Testing
- ✓ Acceptance Testing



The first 2 levels concentrate mainly on the "Functional aspects". During System Testing level, the functionality of the application is tested first, and then the non-functional aspects mainly the "Performance" of the application can be tested.

Acceptance Testing is the last level of testing in the SDLC Apart from the 'Performance' there are other non-functional aspects that can be tested in the application as per the requirements

2.6. *UNIT TESTING*

To test a unit of code (program or set of programs) using Unit Test Specifications, after coding is completed. Involves the basic testing of a piece of code, the size of which is often undefined in practice, although it is usually a function or a subroutine

• Example
– Testing of a cobol program in the reservation system that calculates the price for the ticket requested based on the inputs supplied to the program from the calling cobol program.

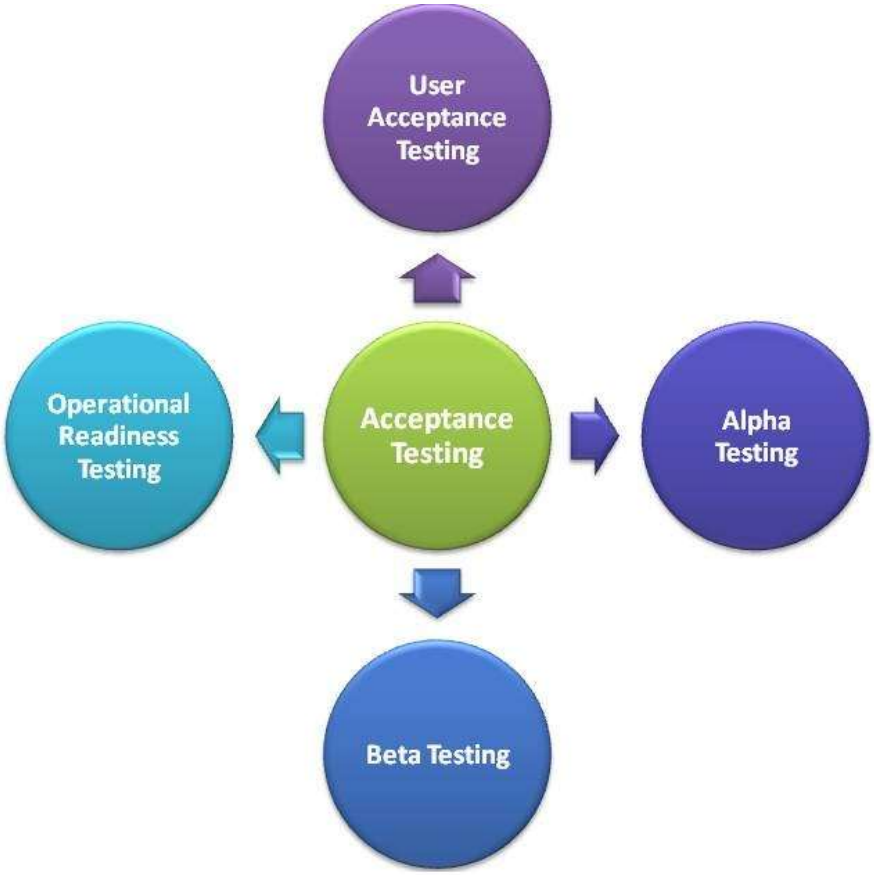
2.7. INTEGRATION TESTING

The process of testing interfaces and data flows between the programs within a sub system,and between the sub-systems within a system. Integration testing means that the tester must look for bugs in the relationship and the interfaces between pairs of components and groups of components under test.
• Example
– Check whether the calling program in previous example passes the right information relating to starting station and destination, date of journey, discounts to be applied and class requested

2.8. *SYSTEM TESTING*

It is a test, executed by the developer or independent test team in a laboratory environment that should demonstrate that the developed system or subsystems meet the requirements set in the functional and quality specifications. The process of proving that the system meets its stated design specifications (design documents) w.r.t criteria such as recoverability, maintainability and security.

2.9. ACCEPTANCE TESTING



It is a test, executed by the user(s) and system manager(s) in an environment simulating the operational environment to the greatest possible extent, that should demonstrate that the developed system meets the functional and quality requirements.

2.10. *NON FUNCTIONAL TESTING*

- ✓ Performance testing
- ✓ Volume testing
- ✓ Load testing
- ✓ Limit testing
- ✓ Stress testing
- ✓ Disaster Testing
- ✓ Recovery testing
- ✓ Security testing
- ✓ Reliability testing
- ✓ Installation Testing
- ✓ Usability Testing

- ✓ Accessibility Testing
- ✓ Regression testing

a) Performance testing:

Performance testing is the process of determining the speed or effectiveness of a computer, network, software program or device. This process can involve quantitative tests done in a lab, such as measuring the response time or the number of MIPS (millions of instructions per second) at which a system functions. Qualitative attributes such as reliability, scalability and interoperability may also be evaluated. Performance testing is often done in conjunction with stress testing. Performance testing can verify that a system meets the specifications claimed by its manufacturer or vendor. The process can compare two or more devices or programs in terms of parameters such as speed, data transfer rate, bandwidth, throughput, efficiency or reliability. Performance testing can also be used as a diagnostic aid in locating communications bottlenecks. Often a system will work much better if a problem is resolved at a single point or in a single component. For example, even the fastest computer will function poorly on today's Web if the connection occurs at only 40 to 50 Kbps (kilobits per second). Slow data transfer rate may be inherent in hardware but can also result from software-related problems, such as:

- Too many applications running at the same time
- A corrupted file in a Web browser
- A security exploit
- Heavy-handed antivirus software
- The existence of active malware on the hard disk.

b) Volume Testing:

- Volume Testing is a type of non-functional testing.
- Volume testing refers to testing a software application or the product with a certain amount of data. E.g., if we want to volume test our application with a specific database size, we need to expand our database to that size and then test the application's performance on it.
- "Volume testing" is a term given and described in Glenford Myers' The Art ofSoftware Testing, 1979. Here's his definition: "Subjecting the program to heavy volumes of data. The purpose of volume testing is to show that the program cannot handle the volume of data specified in its objectives"– p. 113.
- The purpose of volume testing is to determine system performance with increasing volumes of data in the database.

c) Load testing:

In the software development process, the term load testing is often used interchangeably with other forms of testing such as performance testing, volume testing and reliability testing. In simple

words, load testing can be considered the simplest form of performance testing. In load testing, a system or a component is subject to varying load conditions, which are sometimes well beyond the normal limits, in order to determine the behavior of the system at peak load. This process is referred to as stress testing.

Load testing technique may be used in the following scenarios:

- Testing an e-commerce website's shopping cart capacity
- Testing hard disk drive capability to read and write as per its specifications
- Testing an email server to handle email traffic

Load testing helps to find out the maximum amount of load an application can withstand. The success criterion of load testing is based on the completion of all the test cases without any errors and within the allotted time frame. Both load and performance testing are used to analyze software by subjecting it to varying amounts of load while tracking the performance under different load conditions.

d) Stress Testing:

Stress testing refers to the testing of software or hardware to determine whether its performance is satisfactory under any extreme and unfavorable conditions, which may occur as a result of heavy network traffic, process loading, underclocking, overclocking and maximum requests for resource utilization.

Most systems are developed under the assumption of normal operating conditions. Thus, even if a limit is crossed, errors are negligible if the system undergoes stress testing during development.

e) Disaster Testing

This is a process of verifying the success of the restoration procedures that are executed after a critical IT failure or disruption occurs. This could include the following testing;

- What happens to the workload if some of the infrastructure for whatever reason becomes unavailable?
- How long does it take to recover data if a corruption or data loss occurs?
- How does the application cope if part of the network goes down?

This type of testing is sometimes carried out in conjunction with Load Testing. Disaster Recovery Testing is much more realistic if the tests are carried out while the application is busy servicing a user workload.

f) Recovery Testing:

In software testing,recovery testing is the activity of testing how well an application is able to recover from crashes, hardware failures and other similar problems.

Recovery testing is the forced failure of the software in a variety of ways to verify that recovery is

properly performed. Recovery testing should not be confused with reliability testing, which tries to discover the specific point at which failure occurs.Recovery testing is basically done in order to check how fast and better the application can recover against any type of crash or hardware failure etc. Type or extent of recovery is specified in the requirement specifications. It is basically testing how well a system recovers from crashes, hardware failures, or other catastrophic problems

Examples of recovery testing:
1. While an application is running, suddenly restart the computer, and afterwards check the validness of the application's data integrity.
2. While an application is receiving data from a network, unplug the connecting cable. After some time, plug the cable back in and analyze the application's ability to continue receiving data from the point at which the network connection disappeared.
3. Restart the system while a browser has a definite number of sessions. Afterwards, check that the browser is able to recover all of them.

g) Security Testing:

Security testing is a process intended to reveal flaws in the security mechanisms of an information system that protect data and maintain functionality as intended. Due to the logical limitations of security testing, passing security testing is not an indication that no flaws exist or that the system adequately satisfies the security requirements.
Typical security requirements may include specific elements of confidentiality, integrity, authentication, availability, authorization and non-repudiation. Actual security requirements tested depend on the security requirements implemented by the system. Security testing as a term has a number of different meanings and can be completed in a number of different ways. As such a Security Taxonomy helps us to understand these different approaches and meanings by providing a base level to work from.

h) Reliability testing:

A system's reliability is a measure of stability and overall performance of a system collated during an extended period of time under various specific sets of test conditions. This type of testing incorporates the results from non-functional testing such as stress testing, security testing, network testing, along with functional testing. It is a combined metric to define a system's overall reliability. A measure of reliability should be defined by business requirements in the form of service levels. These requirements should then be used to measure test results and the overall reliability metric of a system under test.

i) Installation Testing:

Installation testing is a kind of quality assurance work in the software industry that focuses on

what customers will need to do to install and set up the new software successfully. The testing process may involve full, partial or upgrades install/uninstall processes.
This testing is typically done by the software testing engineer in conjunction with the configuration manager. Implementation testing is usually defined as testing which places a compiled version of code into the testing or pre-production environment, from which it may or may not progress into production. This generally takes place outside of the software development environment to limit code corruption from other future or past releases (or from the use of the wrong version of dependencies such as shared libraries) which may reside on the development environment.

j) Usability Testing:

- In usability testing basically the testers tests the ease with which the user interfaces can be used.

It tests that whether the application or the product built is user-friendly or not.

- Usability Testing is a black box testingtechnique.
- Usability testing also reveals whether users feel comfortable with your application or Web site

according to different parameters - the flow, navigation and layout, speed and content - especially in comparison to prior or similar applications.

- Usability Testing tests the following features of the software.

– How easy it is to use the software.
– How easy it is to learn the software.
– How convenient is the software to end user.

Usability testing includes the following five components:
Learnability: How easy is it for users to accomplish basic tasks the first time they encounter the design?
Efficiency: How fast can experienced users accomplish tasks?
Memorability: When users return to the design after a period of not using it, does the user remember enough to use it effectively the next time, or does the user have to start over again learning everything?
Errors: How many errors do users make, how severe are these errors and how easily can they recover from the errors?
Satisfaction: How much does the user like using the system?

k) Accessibility Testing:

Accessibility testing is the technique of making sure that your product is accessibility compliant. There could be many reasons why your product needs to be accessibility compliant as stated

above.

Typical accessibility problems can be classified into following four groups, each of them with different access difficulties and issues:

Visual impairments

Such as blindness, low or restricted vision, or color blindness. User with visual impairments uses assistive technology software that reads content loud. User with weak vision can also make text larger with browser setting or magnificent setting of operating system.

Motor skills

Such as the inability to use a keyboard or mouse, or to make fine movements.

Hearing impairments

Such as reduced or total loss of hearing

Cognitive abilities

Such as reading difficulties, dyslexia or memory loss.

l) Regression testing:

During confirmation testing the defect got fixed and that part of the application started working as intended. But there might be a possibility that the fix may have introduced or uncovered a different defect elsewhere in the software. The way to detect these '**unexpected side-effects'** of fixes is to do regression testing. The purpose of a regression testing is to verify that modifications in the software or the environment have not caused any unintended adverse side effects and that the system still meets its requirements. Regression testing are mostly automated because in order to fix the defect the same test is carried out again and again and it will be very tedious to do it manually. Regression tests are executed whenever the software changes, either as a result of fixes